

Chapter 10

Artificial Neural Networks

"I think the brain is essentially a computer and consciousness is like a computer program. It will cease to run when the computer is turned off. Theoretically, it could be re-created on a neural network, but that would be very difficult, as it would require all of one's memories."

— **Stephen Hawking**

Artificial Neural Networks (ANNs) imitate human brain behaviour and the way in which learning happens in a human. The human brain constitutes a mass of neurons that are all connected as a network, which is actually a directed graph. These neurons are the processing units which receive information, process it and then transmit this data to other neurons that allows humans to learn almost any task. ANN is a learning mechanism that models a human brain to solve any non-linear and complex problem. Each neuron is modelled as a computing unit, or simply called as a node in ANN, that is capable of doing complex calculations. ANN is a system that consists of many such computing units operating in parallel that can learn from observations. They are widely used in developing artificial learning systems and have inspired researchers and industry in Machine Learning nowadays. Some typical applications of ANN in the field of computer science are Natural Language Processing (NLP), pattern recognition, face recognition, speech recognition, character recognition, text processing, stock prediction, computer vision, etc. ANNs also have been considerably used in other engineering fields such as Chemical industry, Medicine, Robotics, Communications, Banking, and Marketing. This chapter aims to introduce the concepts necessary to understand the working of Artificial Neural Networks.

Learning Objectives

- Understand the basics of human nervous system and biological neurons
- Learn about McCulloch & Pitts Neuron Mathematical Model of an artificial neuron
- Know about the structure of Artificial Neural Network
- Introduce different types of activation functions
- Explore about the first neural network model called 'Perceptron' which is a linear binary classifier used for supervised learning
- Introduce different types of Artificial Neural Networks
- Understand the concepts of learning in Multi-Layer Perceptron (MLP) using back propagation
- Study about Radial Basis Function Neural Network (RBFNN) which is a type of multi-layer perceptron particularly useful for interpolation, function approximation, time series prediction, classification and system control
- Explore Self Organizing Feature Map (SOFM) which is a competitive learning network for unsupervised learning

10.1 INTRODUCTION

The human nervous system has billions of neurons that are the processing units which make humans to perceive things, to hear, to see and to smell. The human nervous system works beautifully, making us understand who we are, what we do, where we are and everything in our surrounding. It makes us to remember, recognize and correlate things around us. It is a learning system that consists of functional units called nerve cells, typically called as neurons. The human nervous system is divided into two sections called the Central Nervous System (CNS) and the Peripheral Nervous System (PNS). The brain and the spinal cord constitute the CNS and the neurons inside and outside the CNS constitute the PNS. The neurons are basically classified into three types called sensory neurons, motor neurons and interneurons. Sensory neurons get information from different parts of the body and bring it into the CNS, whereas motor neurons receive information from other neurons and transmit commands to the body parts. The CNS consists of only interneurons which connect one neuron to another neuron by receiving information from one neuron and transmitting it to another. The basic functionality of a neuron is to receive information, process it and then transmit it to another neuron or to a body part.

Scan for information on *'Convolution Neural Network', Modular Neural Network', and 'Recurrent Neural Network'*

10.2 BIOLOGICAL NEURONS

A typical biological neuron has four parts called dendrites, soma, axon and synapse. The body of the neuron is called as soma. Dendrites accept the input information and process it in the cell body called soma. A single neuron is connected by axons to around 10,000 neurons and through these axons the processed information is passed from one neuron to another neuron. A neuron gets fired if the input information crosses a threshold value and transmits signals to another neuron through a synapse. A synapse gets fired with an electrical impulse called spikes which are transmitted to another neuron. A single neuron can receive synaptic inputs from one neuron or multiple neurons. These neurons form a network structure which processes input information and gives out a response. The simple structure of a biological neuron is shown in Figure 10.1.

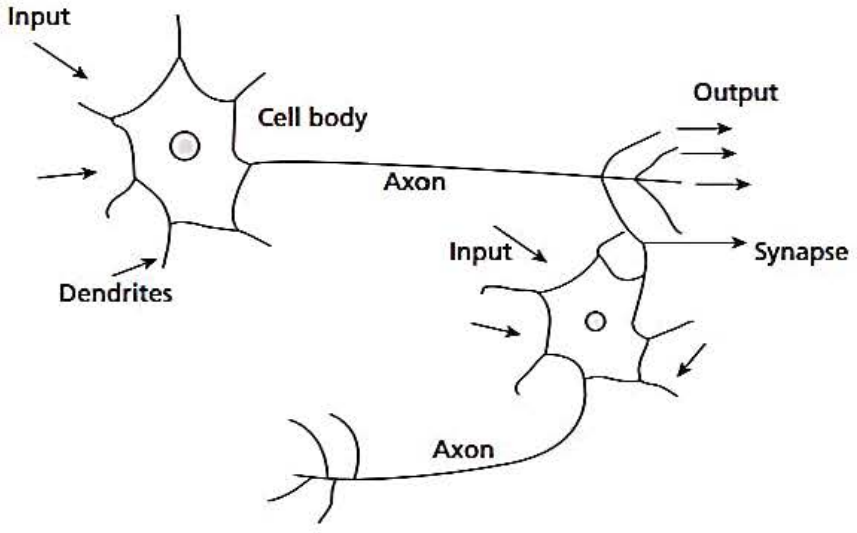


Figure 10.1: A Biological Neuron

10.3 ARTIFICIAL NEURONS

Artificial neurons are like biological neurons which are called as nodes. A node or a neuron can receive one or more input information and process it. Artificial neurons or nodes are connected by connection links to one another. Each connection link is associated with a synaptic weight. The structure of a single neuron is shown in Figure 10.2.

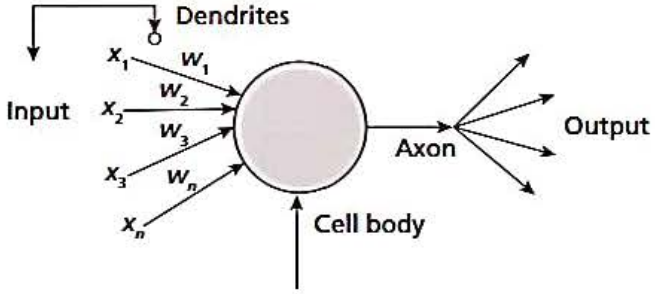


Figure 10.2: An Artificial Neuron

10.3.1 Simple Model of an Artificial Neuron

The first mathematical model of a biological neuron was designed by McCulloch & Pitts in 1943. It includes two steps:

1. It receives weighted inputs from other neurons
2. It operates with a threshold function or activation function

The received inputs are computed as a weighted sum which is given to the activation function and if the sum exceeds the threshold value the neuron gets fired. The mathematical model of a neuron is shown in Figure 10.3.

The neuron is the basic processing unit that receives a set of inputs $x_1, x_2, \ldots, x_n$ and their associated weights $w_1, w_2, \ldots, w_n$. The Summation function 'Net-sum' Eq. (10.1) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^{n} x_i w_i \tag{10.1}$$

The activation function is a binary step function which outputs a value 1 if the Net-sum is above the threshold value θ, and a 0 if the Net-sum is below the threshold value θ. Therefore, the activation function is applied to Net-sum as shown in Eq. (10.2).

$$f(x) = \text{Activation function } (Net - sum) \tag{10.2}$$

$$\text{Then, output of a neuron } Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \tag{10.3}$$

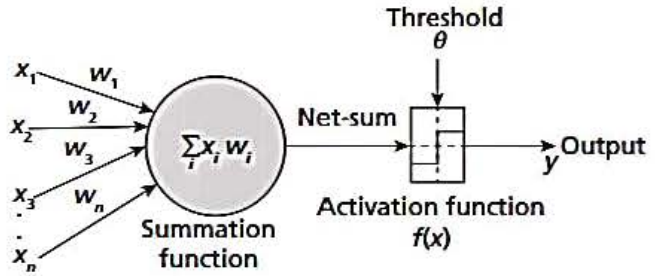


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

McCulloch & Pitts Neuron model can represent only a few Boolean functions. A Boolean function has binary inputs and provides a binary output. For example, an AND Boolean function neuron would fire when all the inputs are 1, whereas an OR Boolean function neuron would fire even when one input is 1. Moreover, the weight and threshold values are fixed in this mathematical model.

10.3.2 Artificial Neural Network Structure

Artificial Neural Network (ANN) imitates a human brain which inhibits some intelligence. It has a network structure represented as a directed graph with a set of neuron nodes and connection links or edges connecting the nodes as shown in Figure 10.4. The nodes in the graph are arrayed in a layered manner and can process information in parallel. The network given in the figure has three layers called input layer, hidden layer and output layer. The input layer receives the input information (x_1, x_2, ..., x_n) and passes it to the nodes in the hidden layer. The edges connecting the nodes from the input layer to the hidden layer are associated with synaptic weights called as connection weights. These computing nodes or neurons perform some computations based on the input information (x_1, x_2, ..., x_n) received and if the weighted sum of the inputs to a neuron is above the threshold or the activation level of the neuron, then the neuron fires. Each neuron employs an activation function that determines the output of the neuron. The neuron transforms linearly the input signals by computing the sum of the product of input signals and weights and adds biases to it. Then, the activation function maps the weighted input sum to a non-linear output value. The node in the output layer gives the output as a single value.

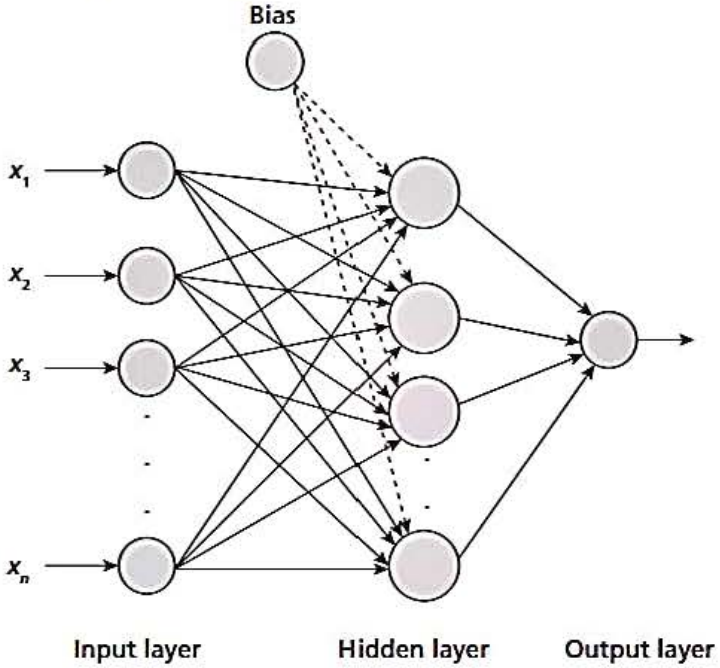


Figure 10.4: Artificial Neural Network Structure

10.3.3 Activation Functions

Activation functions are mathematical functions associated with each neuron in the neural network that map input signals to output signals. It decides whether to fire a neuron or not based on the input signals the neuron receives. These functions normalize the output value of each neuron either between 0 and 1 or between –1 and +1. Typical activation functions can be linear or non-linear.

Linear functions are useful when the input values can be classified into any one of the two groups and are generally used in binary perceptrons. Non-linear functions, on the other hand, are continuous functions that map the input in the range of (0, 1) or (–1, 1), etc. These functions are useful in learning high-dimensional data or complex data such as audio, video and images.

Below are some of the activation functions used in ANNs:

1. Identity Function or Linear Function

$$f(x) = x \ \forall x \tag{10.4}$$

The value of $f(x)$ increases linearly or proportionally with the value of x. This function is useful when we do not want to apply any threshold. The output would be just the weighted sum of input values. The output value ranges between $-\infty$ and $+\infty$.

2. Binary Step Function

$$f(x) = \begin{cases} 1 & if\ f(x) \geq \theta \\ 0 & if\ f(x) < \theta \end{cases} \tag{10.5}$$

The output value is binary, i.e., 0 or 1 based on the threshold value θ. If value of $f(x)$ is greater than or equal to θ, it outputs 1 or else it outputs 0.

3. Bipolar Step Function

$$f(x) = \begin{cases} 1 & if\ f(x) \geq \theta \\ -1 & if\ f(x) < \theta \end{cases} \tag{10.6}$$

The output value is bipolar, i.e., +1 or –1 based on the threshold value θ. If value of $f(x)$ is greater than or equal to θ, it outputs +1 or else it outputs –1.

4. Sigmoidal Function or Logistic Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{10.7}$$

It is a widely used non-linear activation function which produces an *S*-shaped curve and the output values are in the range of 0 and 1. It has a vanishing gradient problem, i.e., no change in the prediction for very low input values and very high input values.

5. Bipolar Sigmoid Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \tag{10.8}$$

It outputs values between –1 and +1.

6. Ramp Functions

$$f(x) = \begin{cases} 1 & if\ x > 1 \\ x & if\ 0 \leq x \leq 1 \\ 0 & if\ x < 0 \end{cases} \tag{10.9}$$

It is a linear function whose upper and lower limits are fixed.

7. Tanh – Hyperbolic Tangent Function

The Tanh function is a scaled version of the sigmoid function which is also non-linear. It also suffers from the vanishing gradient problem. The output values range between –1 and 1.

$$\tan h(x) = \frac{2}{1 + e^{-2x}} - 1 \tag{10.10}$$

8. ReLu – Rectified Linear Unit Function

This activation function is a typical function generally used in deep learning neural network models in the hidden layers. It avoids or reduces the vanishing gradient problem. This function outputs a value of 0 for negative input values and works like a linear function if the input values are positive.

$$r(x) = \max(0, x) = \begin{cases} x & if\ x \geq 0 \\ 0 & if\ x < 0 \end{cases} \tag{10.11}$$

9. Softmax Function

This is a non-linear function used in the output layer that can handle multiple classes. It calculates the probability of each target class which ranges between 0 and 1. The probability of the input belonging to a particular class is computed by dividing the exponential of the given input value by the sum of the exponential values of all the inputs.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{k} e^{x_i}} \text{ where } i = 0 \ldots k \tag{10.12}$$

10.4 PERCEPTRON AND LEARNING THEORY

The first neural network model 'Perceptron', designed by Frank Rosenblatt in 1958, is a linear binary classifier used for supervised learning. He modified the McCulloch & Pitts Neuron model by combining two concepts, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights. He introduced variable weight values and an extra input that represents *bias* to this model. He proposed that artificial neurons could actually learn weights and thresholds from data and came up with a supervised learning algorithm that enabled the artificial neurons to learn the correct weights from training data by itself. The perceptron model (shown in Figure 10.5) consists of 4 steps:

1. Inputs from other neurons
2. Weights and bias
3. Net sum
4. Activation function

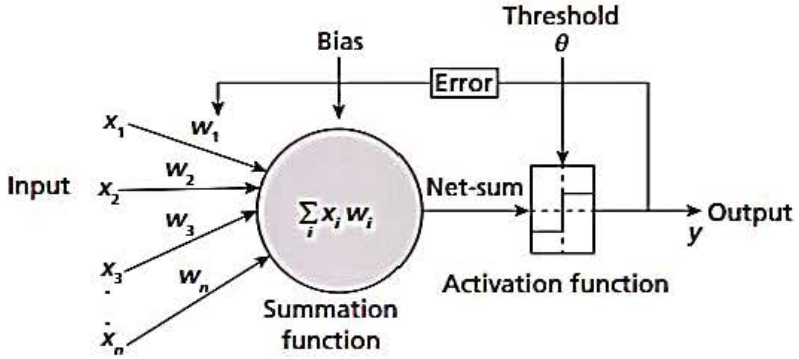


Figure 10.5: Perceptron Model

Thus, the modified neuron model receives a set of inputs $x_1, x_2, \ldots, x_n$, their associated weights $w_1, w_2, \ldots, w_n$ and a bias. The summation function 'Net-sum' Eq. (10.13) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^{n} x_i w_i \tag{10.13}$$

After computing the 'Net-sum', bias value is added to it and inserted in the activation function as shown below:

$$f(x) = \text{Activation function } (\textit{Net-sum} + \textit{bias}) \tag{10.14}$$

The activation function is a binary step function which outputs a value 1 if $f(x)$ is above the threshold value θ, and a 0 if $f(x)$ is below the threshold value θ. Then, output of a neuron:

$$Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \tag{10.15}$$

Before learning how a neural network works, let us learn about how a perceptron model works.

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \ldots, w_n$ and bias θ to a random value in the range [–0.5, 0.5].

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:
$$Y = \text{Step } ((x_1 w_1 + x_2 w_2) - \theta)$$
3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{estimated}$ from the desired output $Y_{desired}$:
$$\text{error } e(t) = Y_{desired} - Y_{estimated}$$
[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y.]
5. Update the weights if there is an error:
$$\Delta w_i = \propto \times e(t) \times x_i$$
$$w_i = w_i + \Delta w_i$$
where, x_i is the input value, $e(t)$ is the error at step t, $\propto$ is the learning rate and Δw_i is the difference in weight that has to be added to w_i.

Example 10.1: Consider a perceptron to represent the Boolean function AND with the initial weights $w_1 = 0.3$, $w_2 = -0.2$, learning rate $\propto = 0.2$ and bias $\theta = 0.4$ as shown in Figure 10.6. The activation function used here is the Step function $f(x)$ which gives the output value as binary, i.e., 0 or 1. If value of $f(x)$ is greater than or equal to 0, it outputs 1 or else it outputs 0. Design a perceptron that performs the Boolean function AND and update the weights until the Boolean function gives the desired output.

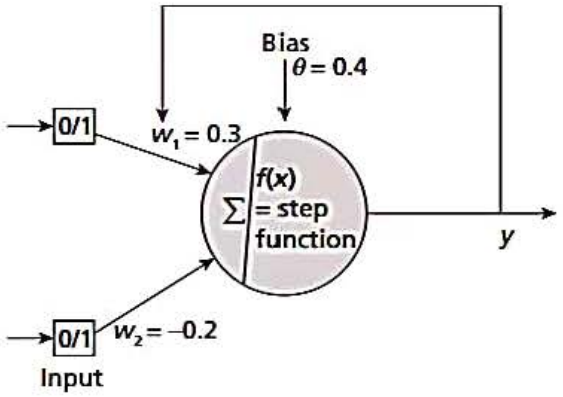


Figure 10.6: Perceptron for Boolean Function AND

Solution: Desired output for Boolean function AND is shown in Table 10.1.

Table 10.1: AND Truth Table

x_1	x_2	Y_{des}
0	0	0
0	1	0
1	0	0
1	1	1

For each Epoch, weighted sum is calculated and the activation function is applied to compute the estimated output Y_{est}. Then, Y_{est} is compared with Y_{des} to find the error. If there is an error, the weights are updated.

Tables 10.2 to 10.5 show how the weights are updated in the four Epochs.

Table 10.2: Epoch 1

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
1	0	0	0	Step $((0 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	–0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	0	0.3	–0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	–0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	1	0.5	0	Change

For input (1, 1) the weights are updated as follows:

$\Delta w_1 = \propto \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$

$w_1 = w_1 + \Delta w_1 = 0.3 + \Delta w_1 = 0.3 + 0.2 = 0.5$

$\Delta w_2 = \propto \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$

$w_2 = w_2 + \Delta w_1 = -0.2 + \Delta w_2 = -0.2 + 0.2 = 0$

Table 10.3: Epoch 2

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
2	0	0	0	Step $((0 \times 0.5 + 0 \times 0) - 0.4) = 0$	0	0.5	0	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0) - 0.4) = 0$	0	0.5	0	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0) - 0.4) = 1$	–1	0.3	0	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0) - 0.4) = 0$	1	0.5	0.2	Change

For input (1, 0) the weights are updated as follows:

$\Delta w_1 = \propto \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$

$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$

$\Delta w_2 = \propto \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$

$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0 = 0$

For input (1, 1), the weights are updated as follows:

$\Delta w_1 = \propto \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$

$w_1 = w_1 + \Delta w_1 = 0.3 + \Delta w_1 = 0.3 + 0.2 = 0.5$

$\Delta w_2 = \propto \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$

$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0.2 = 0.2$

Table 10.4: Epoch 3

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
3	0	0	0	Step $((0 \times 0.5 + 0 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0.2) - 0.4) = 1$	–1	0.3	0.2	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

For input (1, 0) the weights are updated as follows:

$\Delta w_1 = \propto \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$

$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$

$\Delta w_2 = \propto \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$

$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0.2 + 0 = 0.2$

Table 10.5: Epoch 4

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
4	0	0	0	Step $((0 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

It is observed that with 4 Epochs, the perceptron learns and the weights are updated to 0.3 and 0.2 with which the perceptron gives the desired output of a Boolean AND function.

10.4.1 XOR Problem

A perceptron model can solve all Boolean functions which are linearly separable. However, the XOR problem was identified in 1969 by Minsky and Papert. An XOR function returns a 1, if the two inputs are not equal and a 0 if they are equal. Following is the truth table of an XOR function shown in Table 10.6.

Table 10.6: XOR Truth Table

x_1	x_2	Y
0	0	1
0	1	0
1	0	0
1	1	1

Since XOR is not linearly separable, the single layer perceptron failed to classify and hence this problem led to the evolution of a multi-layer perceptron. Initially, the Multi-Layer Perceptron (MLP) was not a success due to the lack of an appropriate learning algorithm. In 1974, Werbos introduced a back propagation algorithm for the three-layered perceptron network and in 1986, a general back propagation algorithm for a multi-layered perceptron was introduced by Rummelhart and Mclelland. Then, again ANN and Deep Neural Networks became a success, solving many complex problems in the current era. Thus, MLP that emerged could solve any non-linear separable problem which is explained later in this chapter.

10.4.2 Delta Learning Rule and Gradient Descent

Generally, learning in neural networks is performed by adjusting the network weights in order to minimize the difference between the desired and estimated outputs. This delta difference is measured as an error function or also called as cost function. The cost function, being linear and continuous, is differentiable. This way of learning called as delta rule (also known as Widrow-Hoff" rule or Adaline rule) is a type of back propagation applied for training the network. The training error of a hypothesis is half the squared difference between the desired target output and actual output and is given as follows:

$$\text{Training Error} = \frac{1}{2}\Sigma_{d\in T}(O_{Desired} - O_{Estimated})^2 \tag{10.16}$$

where, T is the training dataset, $O_{Desired}$ and $O_{Estimated}$ are the desired target output and estimated actual output, respectively, for a training instance d.

The principle of gradient descent is an optimization approach which is used to minimize the cost function by converging to a local minimal point moving in the negative direction of the gradient and each step size during movement is determined by the learning rate and the slope of the gradient.

Gradient descent learning is the foundation of back propagation algorithm used in MLP. Before we study about an MLP, let us first understand the different types of neural networks that differ in their structure, activation function and learning mechanism.

10.5 TYPES OF ARTIFICIAL NEURAL NETWORKS

ANNs consist of multiple neurons arranged in layers. There are different types of ANNs that differ by the network structure, activation function involved and the learning rules used. In an ANN, there are three layers called input layer, hidden layer and output layer. Any general ANN would consist of one input layer, one output layer and zero or more hidden layers.

10.5.1 Feed Forward Neural Network

This is the simplest neural network that consists of neurons which are arranged in layers and the information is propagated only in the forward direction. This model may or may not contain a hidden layer and there is no back propagation. Based on the number of hidden layers they are further classified into single-layered and multi-layered feed forward networks. These ANNs are simple to design and easy to maintain. They are fast but cannot be used for complex learning. They are used for simple classification and simple image processing, etc. The model of a Feed Forward Neural Network is shown in Figure 10.7.

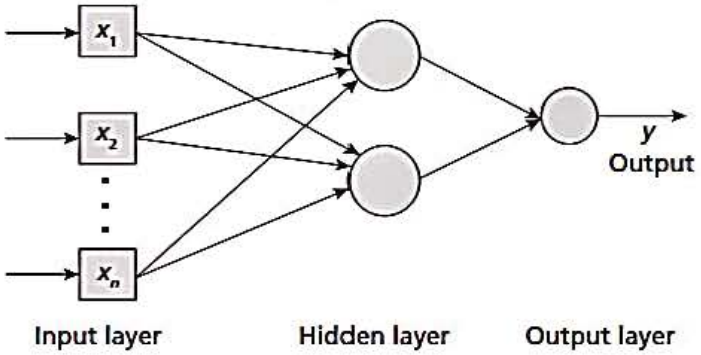


Figure 10.7: Model of a Feed Forward Neural Network

10.5.2 Fully Connected Neural Network

Fully connected neural networks are the ones in which all the neurons in a layer are connected to all other neurons in the next layer. The model of a fully connected neural network is shown in Figure 10.8.

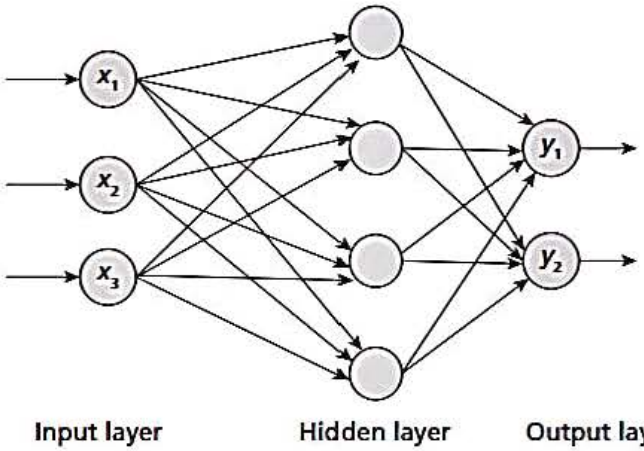


Figure 10.8: Model of a Fully Connected Neural Network

10.5.3 Multi-Layer Perceptron (MLP)

This ANN consists of multiple layers with one input layer, one output layer and one or more hidden layers. Every neuron in a layer is connected to all neurons in the next layer and thus they are fully connected. The information flows in both the directions. In the forward direction, the inputs are multiplied by weights of neurons and forwarded to the activation function of the

neuron and output is passed to the next layer. If the output is incorrect, then in the backward direction, error is back propagated to adjust the weights and biases to get correct output. Thus, the network learns with the training data. This type of ANN is used in deep learning for complex classification, speech recognition, medical diagnosis, forecasting, etc. They are comparatively complex and slow. The model of an MLP is shown in Figure 10.9.

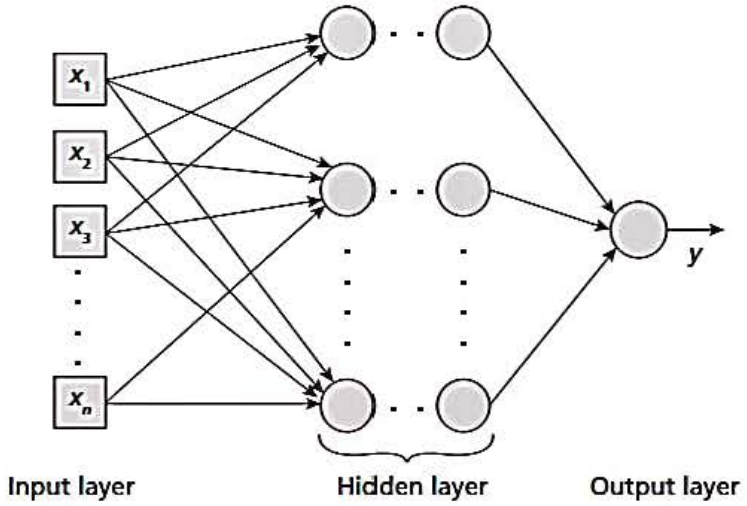


Figure 10.9: Model of a Multi-Layer Perceptron

10.5.4 Feedback Neural Network

Feedback neural networks have feedback connections between neurons that allow information flow in both directions in the network. The output signals can be sent back to the neurons in the same layer or to the neurons in the preceding layers. Hence, this network is more dynamic during training. The model of a feedback neural network is shown in Figure 10.10.

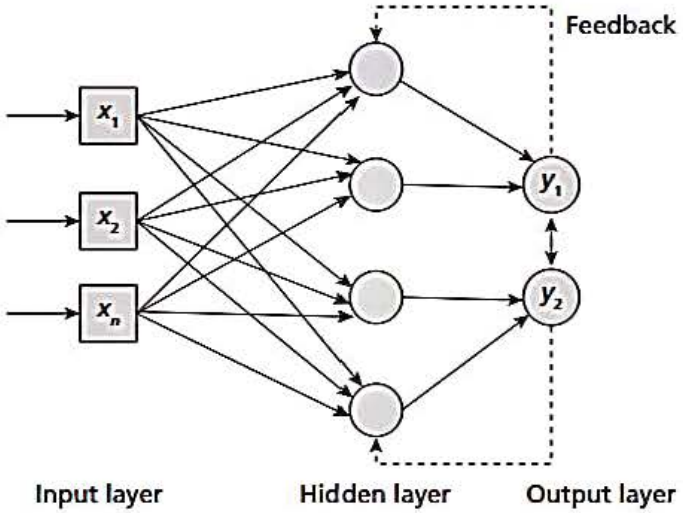


Figure 10.10: Model of a Feedback Neural Network

10.6 LEARNING IN A MULTI-LAYER PERCEPTRON

A multi-layer perceptron is a type of Feed Forward Neural Network with multiple neurons arranged in layers. All the neurons in a layer are fully connected to the neurons in the next layer. The network has atleast three layers with an input layer, one or more hidden layers and

an output layer. The input layer is the visible layer. It just passes the input to the next layer. The layers following the input layer are the hidden layers. The hidden layers neither directly receive inputs nor send outputs to the external environment. The final layer is the output layer which outputs a single value or a vector of values. An MLP has atleast one hidden layer and as the number of hidden layers is increased, the learning becomes more complex and they form a deep neural network. It uses back propagation for supervised learning of the network. The activation functions used in the layers can be linear or non-linear depending on the type of the problem modelled. Typically, a sigmoid activation function is used if the problem is a binary classification problem and a softmax activation function is used in a multi-class classification problem.

The typical non-linear separable problem is the XOR problem which could be solved with an MLP. A single neuron can only classify the values into two categories by drawing a single straight line. However, to classify the XOR inputs, we need two straight lines. Hence, the problem could be solved if we add a hidden layer having two neurons working in parallel in the same layer and combine the outputs to get a single output. One neuron should work as an OR gate and the other neuron should work as a NAND gate. The neuron in the output layer should work as an AND gate.

The MLP network learns with two phases called the forward phase and the backward phase. In the forward phase, an input vector from the training dataset is taken and given to the network which outputs a value called the estimated value $O_{Estimated}$. This value is compared with the desired output value $O_{Desired}$ and the error is calculated. The calculated error is back propagated in the backward phase to update the weights and biases of the network. This process is repeated for the entire training dataset.

The algorithm can be terminated after a pre-defined number of epochs or if the training error is reduced below a threshold value. This way of minimizing the training error by updating the weights and biases may lead to overfitting problem in neural networks. Several techniques are there to overcome this overfitting problem. One successful solution is to provide a set of validation data along with the training data and use a cross validation approach to estimate the number of iterations or weight updates to perform that produces the lowest error with the validation set and stop when the validation error begins to increase.

Algorithm 10.2: Learning in an MLP

Input: Input vector $(x_1, x_2, ..., x_n)$

Output: Y_n

Learning rate: α

Assign random weights and biases for every connection in the network in the range [–0.5, +0.5].

Step 1: Forward Propagation

1. **Calculate Input and Output in the *Input Layer*:**

   (Input layer is a direct transfer function, where the output of the node equals the input).

   Input at Node j 'I$_j$' in the Input Layer is

$$\boxed{I_j = x_j}$$

(*Continued*)

where,

x_j is the input received at Node j

Output at Node j 'O_j' in the Input Layer is

$$O_j = I_j$$

2. **Calculate Net Input and Output in the *Hidden Layer* and *Output Layer*:**

Net Input at Node j in the Hidden Layer is

$$I_i = \sum_{i=1}^{n} x_i w_{ij} + x_0 \times \theta_j$$

where,

x_i is the input from Node i

w_{ij} is the weight in the link from Node i to Node j

x_0 is the input to bias node '0' which is always assumed as 1

θ_j is the weight in the link from the bias node '0' to Node j

Net Input at Node j in the Output Layer is

$$I_j = \sum_{i=1}^{n} O_i w_{ij} + x_0 \times \theta_j$$

where,

O_i is the output from Node i

w_{ij} is the weight in the link from Node i to Node j

x_0 is the input to bias node '0' which is always assumed as 1

θ_j is the weight in the link from the bias node '0' to Node j

Output at Node j

$$O_j = \frac{1}{1 + e^{-Ij}}$$

where,

I_j is the input received at Node j

3. **Estimate error at the node in the *Output Layer*:**

$$\text{Error} = O_{Desired} - O_{Estimated}$$

where,

$O_{Desired}$ is the desired output value of the Node in the Output Layer

$O_{Estimated}$ is the estimated output value of the Node in the Output Layer

Step 2: Backward Propagation

1. **Calculate Error at each node:**

For each Unit k in the Output Layer

$$\text{Error}_k = O_k(1 - O_k)(O_{Desired} - O_k)$$

(Continued

where,

O_k is the output value at Node k in the Output Layer.

$O_{Desired}$ is the desired output value of the Node in the Output Layer.

For each unit j in the Hidden Layer

$$\boxed{\text{Error}_j = O_j(1 - O_j)\ \Sigma_k\ \text{Error}_k\ w_{jk}}$$

where,

O_j is the output value at Node j in the Hidden Layer.

Error_k is the error at Node k in the Output Layer.

w_{jk} is the weight in the link from Node j to Node k.

2. Update all weights and biases:

Update weights

$$\boxed{\begin{aligned}\Delta w_{ij} &= \propto \times \text{Error}_j \times O_i \\ w_{ij} &= w_{ij} + \Delta w_{ij}\end{aligned}}$$

where,

O_i is the output value at Node i.

Error_j is the error at Node j.

$\propto$ is the learning rate.

w_{ij} is the weight in the link from Node i to Node j.

Δw_{ij} is the difference in weight that has to be added to w_{ij}.

Update Biases

$$\boxed{\begin{aligned}\Delta\theta_j &= \propto \times \text{Error}_j \\ \theta_j &= \theta_j + \Delta\theta_j\end{aligned}}$$

where,

Error_j is the error at Node j.

$\propto$ is the learning rate.

θ_j is the bias value from Bias Node 0 to Node j.

$\Delta\theta_j$ is the difference in bias that has to be added to θ_j.

Example 10.2: Consider learning in a Multi-Layer Perceptron. The given MLP consists of an Input layer, one Hidden layer and an Output layer. The input layer has 4 neurons, the hidden layer has 2 neurons and the output layer has a single neuron. Train the MLP by updating the weights and biases in the network.

x_1	x_2	x_3	x_4	$O_{Desired}$
1	1	0	1	1

Learning rate: = 0.8.

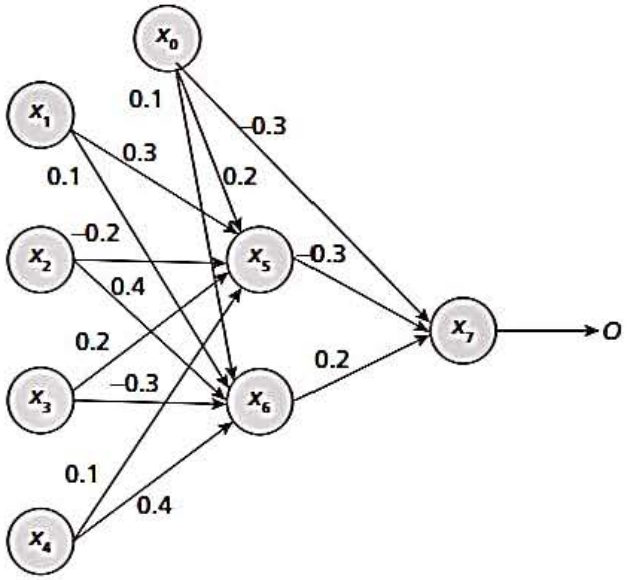


Figure 10.11: Example MLP

Solution: From the Figure 10.11, the weights and biases are tabulated in Table 10.7.

Table 10.7: Weights and Biases

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	−0.2	0.4	0.2	−0.3	0.1	0.4	−0.3	0.2	0.2	0.1	−0.3

Step 1: Forward Propagation

1. Calculate Input and Output in the Input Layer shown in Table 10.8.

Table 10.8: Net Input and Output Calculation

Input layer	I_j	O_j
x_1	1	1
x_2	1	1
x_3	0	0
x_4	1	1

2. Calculate Net Input and Output in the Hidden Layer and Output Layer as shown in Table 10.9.

Table 10.9: Unit j at Hidden Layer and Output Layer – Net Input and Output Calculation

Unit j	Net Input I_j	Output O_j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.3 + 1 \times -0.2 + 0 \times 0.2 + 1 \times 0.1 + 1 \times 0.2 = 0.4$	$O_5 = \frac{1}{1+e^{-I_5}} \quad \frac{1}{1+e^{-0.4}} = 0.599$
x_6	$I_6 = x_1 \times w_{15} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_5 = 1 \times 0.3 + 1 \times 0.4 + 0 \times -0.3 + 1 \times 0.4 + 1 \times 0.1 = 1.2$	$O_6 = \frac{1}{1+e^{-I_6}} \quad \frac{1}{1+e^{-1.2}} = 0.769$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.599 \times -0.3 + 0.769 \times 0.2 + 1 \times -0.3 = -0.326$	$O_7 = \frac{1}{1+e^{-I_7}} \quad \frac{1}{1+e^{0.326}} = 0.419$

3. Calculate Error $= O_{desired} - O_{Estimated}$

   So, error for this network is:

   Error $= O_{desired} - O_7 = 1 - 0.419 = 0.581$

   So, we need to back propagate to reduce the error.

Step 2: Backward Propagation

1. Calculate Error at each node as shown in Table 10.10.

   For each unit k in the output layer, calculate:

   $\text{Error}_k = O_k(1 - O_k)(O_{desired} - O_k)$

   For each unit j in the hidden layer, calculate:

   $\text{Error}_j = O_j(1 - O_j) \sum_k \text{Error}_k w_{jk}$

Table 10.10: Error Calculation for Each Unit in the Output Layer and Hidden Layer

For Output Layer Unit_k	Error_k
x_7	$\text{Error}_7 = O_7(1 - O_7)(Y_n - O_7)$ $= 0.419 \times (1 - 0.419) \times (1 - 0.419) = 0.141$
For Hidden Layer Unit_j	**Error_j**
x_6	$\text{Error}_6 = O_6(1 - O_6) \sum_k \text{Error}_k w_{jk} = O_6(1 - O_6)\,\text{Error}_7 w_{67}$ $= 0.769\,(1 - 0.769) \times 0.2 \times 0.141 = 0.005$
x_5	$\text{Error}_5 = O_5(1 - O_5) \sum_k \text{Error}_k w_{jk} = O_5(1 - O_5)\,\text{Error}_7 w_{57}$ $= 0.599(1 - 0.599) \times 0.141 \times -0.3 = -0.0101$

2. Update weight using the below formula:

   Learning rate $\alpha = 0.8$.

   $\Delta w_{ij} = \propto \times \text{Error}_j \times O_i$

   $w_{ij} = w_{ij} + \Delta w_{ij}$

   The updated weights and bias are shown in Tables 10.11 and 10.12, respectively.

Table 10.11: Weight Updation

w_{ij}	$w_{ij} = w_{ij} + \propto \times \text{Error}_j \times O_i$	New Weight
w_{15}	$w_{15} = w_{15} + 0.8 \times \text{Error}_5 \times O_1$ $= 0.3 + 0.8 \times -0.0101 \times 1$	0.292
w_{16}	$w_{16} = w_{16} + 0.8 \times \text{Error}_6 \times O_1$ $= 0.1 + 0.8 \times 0.005 \times 1$	0.104
w_{25}	$w_{25} = w_{25} + 0.8 \times \text{Error}_5 \times O_2$ $= -0.2 + 0.8 \times -0.0101 \times 1$	–0.208
w_{26}	$w_{26} = w_{26} + 0.8 \times \text{Error}_6 \times O_2$ $= 0.4 + 0.8 \times 0.005 \times 1$	0.404

(Continued)

w_{ij}	$w_{ij} = w_{ij} + \propto \times Error_j \times O_i$	New Weight
w_{35}	$w_{35} = w_{35} + 0.8 \times Error_5 \times O_3$ $= 0.2 + 0.8 \times -0.0101 \times 0$	0.2
w_{36}	$w_{36} = w_{36} + 0.8 \times Error_6 \times O_3$ $= -0.3 + 0.8 \times 0.005 \times 0$	–0.3
w_{45}	$w_{45} = w_{45} + 0.8 \times Error_5 \times O_4$ $= 0.1 + 0.8 \times -0.0101 \times 1$	0.092
w_{46}	$w_{46} = w_{46} + 0.8 \times Error_6 \times O_4$ $= 0.4 + 0.8 \times 0.005 \times 1$	0.404
w_{57}	$w_{57} = w_{57} + 0.8 \times Error_7 \times O_5$ $= -0.3 + 0.8 \times 0.141 \times 0.599$	–0.232
w_{67}	$w_{67} = w_{67} + 0.8 \times Error_7 \times O_6$ $= 0.2 + 0.8 \times 0.141 \times 0.769$	0.287

Update bias using the below formula:

$$\Delta\theta_j = \propto \times Error_j$$
$$\theta_j = \theta_j + \Delta\theta_j$$

Table 10.12: Bias Updation

θ_j	$\theta_j = \theta_j + \propto \times Error_j$	New Bias
θ_5	$\theta_5 = \theta_5 + \propto \times Error_5$ $= 0.2 + 0.8 \times -0.0101$	0.192
θ_6	$\theta_6 = \theta_6 + \propto \times Error_6$ $= 0.1 + 0.8 \times 0.005$	0.104
θ_7	$\theta_7 = \theta_7 + \propto \times Error_7$ $= -0.3 + 0.8 \times 0.141$	–0.187

Iteration 2

Now, with the updated weights and biases:

1. Calculate Input and Output in the Input Layer as shown in Tal

Table 10.13: Net Input and Output Calculatio

Input Layer	I_j	O_j
x_1	1	1
x_2	1	1
x_3	0	0
x_4	1	1

2. Calculate Net Input and Output in the Hidden Layer an Table 10.14.

Table 10.14: Net Input and Output Calculation in the Hidden Layer and Output Layer

Unit j	Net Input I_j	Output O_j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.292 + 1 \times -0.208 + 0 \times 0.2 + 1 \times 0.092 + 1 \times 0.192 = 0.368$	$O_5 = \frac{1}{1+e^{-I_5}} = \frac{1}{1+e^{-0.368}} = 0.591$
x_6	$I_6 = x_1 \times w_{15} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_6 = 1 \times 0.292 + 1 \times 0.404 + 0 \times -0.3 + 1 \times 0.404 + 1 \times 0.104 = 1.204$	$O_6 = \frac{1}{1+e^{-I_6}} = \frac{1}{1+e^{-1.204}} = 0.7692$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.591 \times -0.232 + 0.7692 \times 0.287 + 1 \times -0.187 = -0.326$	$O_7 = \frac{1}{1+e^{-I_7}} = \frac{1}{1+e^{0.1034}} = 0.474$

The output we receive in the network at node 7 is 0.474.

Error = 1 – 0.474 = 0.526

Now, when we compare the error we get in the previous iteration and in the current iteration, it is visible that the network has learnt and reduced the error by 0.055.

Error is reduced by 0.055: 0.581 – 0.526.

Thus, the training is continued for a predefined number of epochs or until the training error is reduced below a threshold value.

10.7 RADIAL BASIS FUNCTION NEURAL NETWORK

Radial Basis Function Neural Network (RBFNN) was introduced by Broomhead and Lowe in 1988. It is a type of Multi Layer Perceptron which has one input layer, one output layer and with strictly one hidden layer. The hidden layer uses a non-linear radial basis function as the activation function, which converts the input parameters into high dimension space which is then fed into the network to linearly separate the problem. An XOR function is not linearly separable and requires at least one hidden layer to classify it. The RBFNN uses the hidden layer to derive the feature vector whose dimension is increased in space. This neural network is useful for interpolation, function approximation, time series prediction, classification and system control.

Typical Radial Basis Functions (RBF) are:

The Gaussian RBF which monotonically decreases with distance from the centre.

$$H(x) = e^{\frac{-(x-c)^2}{r^2}} \tag{10.17}$$

where, c is the centre and r is the radius.

A Multiquadric RBF which monotonically increases with distance from the centre.

$$H(x) = \sqrt{\frac{r^2 + (x-c)^2}{r}} \tag{10.18}$$

RBFNN architecture includes:

1. An input layer that feeds the input vector of n-dimension to the network ($x_1, x_2, ..., x_n$).
2. A hidden layer that comprises 'm' non-linear radial basis function neurons where $m \geq n$. The hidden layer implements the Radial Basis Function called Gaussian function. The output of a hidden layer neuron for an input vector x is given as in Eq. (10.17):

$$H(x) = e^{\frac{-(x-c)^2}{r^2}}$$

where, x is the input vector, c is the centre and r is the radius.

Each RBF neuron in the hidden layer compares the input vector with the centre of the neuron which is a bell curve and outputs a similarity value between 0 and 1. If the input is equal to the neuron centre, then the output is 1 but as the difference increases, the activation value or the output of the neuron falls off exponentially towards 0.

3. An output layer that computes the linear weighted sum of the output of each neuron from the hidden layer neurons:

$$F(x) = \sum_{i=1}^{m} w_i H_i(x) \tag{10.19}$$

where,

w_i is the weight in the link from the Hidden Layer neuron i to the Output Layer.

$H_i(x)$ is the output of a Hidden Layer neuron i for an input vector x.

The architecture of a Radial Basis Function Neural Network is shown in Figure 10.12.

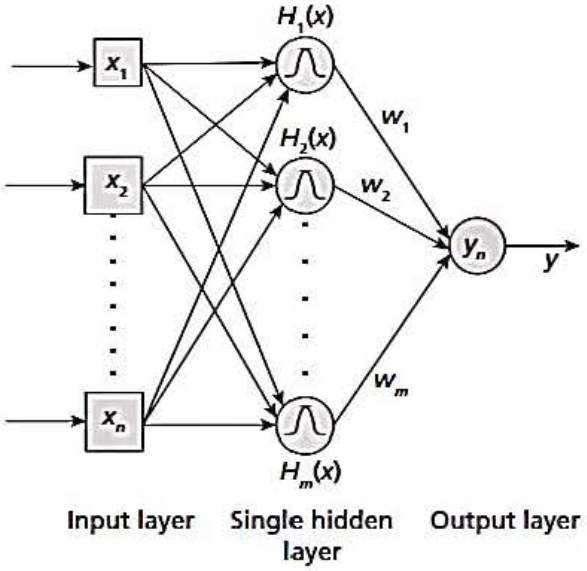


Figure 10.12: Architecture of RBFNN

Training or learning with RBFNN is very fast and the neural network is very good at interpolation.

Algorithm 10.3: Radial Basis Function Neural Network

Input: Input vector $(x_1, x_2, ..., x_n)$

Output: Y_n

Assign random weights for every connection from the Hidden layer to the Output layer in the network in the range [–1, +1].

Forward Phase:

Step 1: Calculate Input and Output in the Input Layer:

(Input layer is a direct transfer function, where the output of the node equals the input).

Input at Node i 'I$_i$' in the Input Layer is

$$\boxed{I_i = x_i}$$

where,

x_i is the input received at Node i.

(*Continued*)

Output at Node i 'O_i' in the Input Layer is

$$\boxed{O_i = I_i}$$

Step 2: For each node j in the Hidden Layer, find the centre/receptor c and the variance r.

Define hidden layer neurons with Gaussian RBF whose output is:

$$\boxed{H_j(x) = e^{\frac{-(x-c_j)^2}{r^2}}}$$

where, x is the input, c_j is the centre and r is the radius.

Compute $(x - c_j)^2$ applying Euclidean distance measure between x and c_j.

Step 3: For each node k in the Output Layer, compute linear weighted sum of the output of each neuron k from the hidden layer neurons j.

$$\boxed{F_k(x) = \sum_{j=1}^{m} w_{jk} H_j(x)}$$

where,

w_{jk} is the weight in the link from the Hidden Layer neuron j to the Output Layer neuron k.

$H_j(x)$ is the output of a Hidden Layer neuron j for an input vector x.

Backward Phase:

Step 1: Train the Hidden layer using Back propagation.

Step 2: Update the weights between the Hidden layer and Output layer.

Example 10.3: Consider the XOR Boolean function that has 4 patterns (0, 0) (0, 1) (1, 0) and (1, 1) in a 2-dimensional input space.

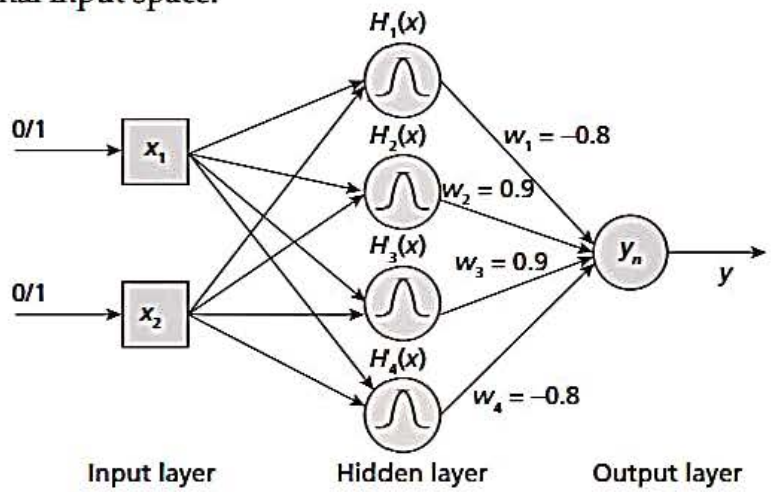


Figure 10.13: Example RBFNN

Construct a RBFNN as shown in Figure 10.13 that classifies the input pattern:

(0, 0) → 0

(0, 1) → 1

(1, 0) → 1

(1, 1) → 0

Solution: Define 4 hidden layer neurons with Gaussian RBF:

$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}}$; $c1 = (0, 0)$

$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}}$; $c2 = (0, 1)$

$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}}$; $c3 = (1, 0)$

$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}}$; $c4 = (1, 1)$

For input pattern (0, 0)

Distance squared of x from $c1 = (0, 0)$
$= (0-0)^2 + (0-0)^2 = 0$

$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$

Distance squared of x from $c2 = (0, 1)$
$= (0-0)^2 + (0-1)^2 = 1$

$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c3 = (1, 0)$
$= (0-1)^2 + (0-0)^2 = 1$

$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c4 = (1, 1)$
$= (0-1)^2 + (0-1)^2 = 2$

$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 1.0 + 0.9 \times 0.6 + 0.9 \times 0.6 + -0.8 \times 0.4 = -0.04$

For input pattern (0, 1)

Distance squared of x from $c1 = (0, 0)$
$= (0-0)^2 + (1-0)^2 = 1$

$H_1(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c2 = (0, 1)$
$= (0-0)^2 + (1-1)^2 = 0$

$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 1.0$

Distance squared of x from $c3 = (1, 0)$
$= (0-1)^2 + (1-0)^2 = 2$

$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$

Distance squared of x from $c4 = (1, 1)$
$= (0-1)^2 + (1-1)^2 = 1$

$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 1.0 + 0.9 \times 0.4 + -0.8 \times 0.6 = 0.3$

For input pattern (1, 0)

Distance squared of x from $c1 = (0, 0)$
$= (1-0)^2 + (0-0)^2 = 1$

$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c2 = (0, 1)$
$= (1-0)^2 + (0-1)^2 = 2$

$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$

Distance squared of x from $c3 = (1, 0)$
$= (1-1)^2 + (0-0)^2 = 0$

$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 1.0$

Distance squared of x from $c4 = (1, 1)$
$= (1-1)^2 + (0-1)^2 = 1$

$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 0.4 + 0.9 \times 1.0 + -0.8 \times 0.6 = 0.3$

For input pattern (1, 1)

Distance squared of x from $c1 = (0, 0)$
$= (1-0)^2 + (1-0)^2 = 2$

$H_1(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$

Distance squared of x from $c2 = (0, 1)$
$= (1-0)^2 + (1-1)^2 = 1$

$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c3 = (1, 0)$
$= (1-1)^2 + (1-0)^2 = 1$

$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$

Distance squared of x from $c4 = (1, 1)$
$= (1-1)^2 + (1-1)^2 = 0$

$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.4 + 0.9 \times 0.6 + 0.9 \times 0.6 + -0.8 \times 1.0 = -0.04$

Table 10.15 shows the obtained values during the calculation done in the forward phase.

Table 10.15: Forward Phase Calculation

Input		$H_1(x)$	$H_2(x)$	$H_3(x)$	$H_4(x)$	$\sum_{j=1}^{m} w_j H_j(x)$	Output
0	0	1.0	0.6	0.6	0.4	–0.04	0
0	1	0.6	1.0	0.4	0.6	0.3	1
1	0	0.6	0.4	1.0	0.6	0.3	1
1	1	0.4	0.6	0.6	1.0	–0.04	0
		$w_1 = -0.8$	$w_2 = 0.9$	$w_3 = 0.9$	$w_4 = -0.8$		

RBF Networks are generally trained to determine the following parameters:

1. The number of neurons in the hidden layer
2. The center of each hidden-layer RBF neuron
3. The radius or variance of each RBF function
4. The weights assigned from the Hidden layer to the Output layer for the summation function

Different approaches are followed to determine the centres for the Hidden layer RBF neurons, comprising:

1. Random selection of fixed cluster centres
2. Self-organized selection of centres using *K*-means clustering
3. Supervised selection of centres

10.8 SELF-ORGANIZING FEATURE MAP

Self-Organizing Feature Map (SOFM) is a special type of Feed Forward Artificial Neural Network developed by Dr Teuvo Kohonen in 1982. Kohonen network is a competitive learning network or also called as adaptive learning network. SOFM is an unsupervised learning model that clusters data by mapping a high-dimensional data into a two-dimensional map (neurons) or plane. The model learns to cluster or self organize a high-dimensional data without knowing the class membership of the input data, and hence the name self-organizing nodes. These self-organizing nodes are also called as feature maps. The mapping is based on the relative distance or similarity between the points and the points that are near to each other in the input space are mapped to nearby output map units in the SOFM.

Network Architecture and Operations

The network architecture consists of only two layers called the Input layer and the Output layer, and there are no Hidden layers. The number of units in the Input layer is based on the length of the input samples which is a vector of length '*n*'. Each connection from the Input units in the Input layer to the output units in the Output layer is assigned with random weights. There is one weight vector of length '*n*' associated with each output unit. Output units have intra layer connections with no weights assigned between these connections but used for updating the weights. The network architecture of SOFM is shown in Figure 10.14.

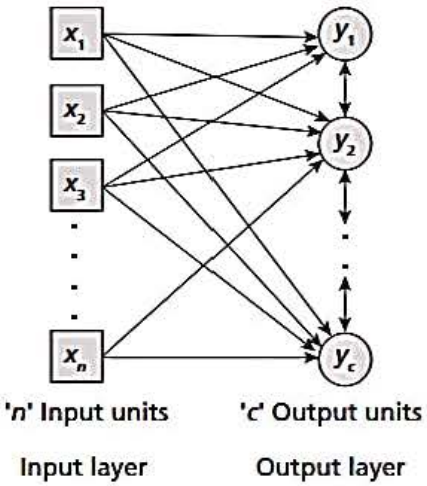


Figure 10.14: Network Architecture of Self Organizing Feature Map

The SOFM network operates in two phases, the training phase and the mapping phase. During the training phase, the input layer is fed with input samples randomly from the training data. The units or neurons in the output layer are initially assigned with some weights. As the input sample is fed, each output unit computes a similarity score by Euclidean distance measure and compete with each other. The output unit, which is close to the input sample by similarity, is chosen as the winning unit and its connection weights are adjusted by a learning factor. Thus, the best matching output unit whose weights are adjusted are moved close to the input sample and a topological feature map is formed. This process is repeated until the map does not change. During the mapping phase, the test samples are just classified.

Algorithm 10.4: Self-Organizing Feature Map

Input: 'm' Training Vectors ($x_1, x_2, \ldots, x_m$), each of length 'n':

$$\begin{bmatrix} x_{1,1} & x_{1,2} \ldots . x_{1,n} \\ x_{2,1} & x_{2,2} \ldots . . x_{2,n} \\ & \\ & \\ x_{m,1} & x_{m,2} \ldots . . x_{m,n} \end{bmatrix}$$

Output: 'c' categories

Input Layer: 'n' Input units (Length of the training vector)

Output Layer: 'c' Output units (Number of categories)

Step 1: Initialize random weights w_j between 0 and 1, for each Output unit j.

(A Weight vector of length 'n' is associated with each Output unit)

Step 2: For each training sample x_s:

(a) Compute Euclidean distance score between the training sample x_s and the weight vector w_j of each output unit:

(*Continued*)

$$d^2 = (\text{Euclidean distance})^2 = \sum_{k=1}^{n}(x_{s,k} - w_{j,k}(t))^2$$

(b) Choose the winning output unit which has smaller distance to the input sample as the best matching unit.

(c) Update the weights for the winning unit:

$$w_j(t+1) = w_j(t) + \eta(t)(x_s - w_j(t))$$

where,

$\eta(t)$ is the Learning rate.

$w_j(t)$ is the weight of the winning unit at step t.

$w_j(t+1)$ is the updated weight of the winning unit at step $(t+1)$.

Step 3: Repeat until the feature map doesn't change.

Example 10.4: Consider the example shown in Figure 10.15 which considers four training samples each vector of length 4 and two output units. Train the SOFM network by determining the class memberships of the input data.

Training Samples:

x_1: (1, 0, 1, 0)

x_2: (1, 0, 0, 0)

x_3: (1, 1, 1, 1)

x_4: (0, 1, 1, 0)

Output Units: Unit 1, Unit 2

Learning rate $\eta(t) = 0.6$

Initial Weight matrix

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix} : \begin{bmatrix}0.3 & 0.5 & 0.7 & 0.2\\ 0.6 & 0.5 & 0.4 & 0.2\end{bmatrix}$$

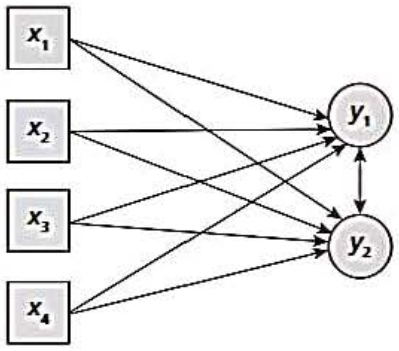


Figure 10.15: Example SOFM

Solution:

Iteration 1:

Training Sample x_1: (1, 0, 1, 0)

Weight matrix:

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.3 & 0.5 & 0.7 & 0.2\\ 0.6 & 0.7 & 0.4 & 0.3\end{bmatrix}$$

Compute Euclidean distance between x_1: (1, 0, 1, 0) and Unit 1 weights.

$$d^2 = (0.3-1)^2 + (0.5-0)^2 + (0.7-1)^2 + (0.2-0)^2 = 0.87$$

Compute Euclidean distance between x_1: (1, 0, 1, 0) and Unit 2 weights.

$$d^2 = (0.6-1)^2 + (0.7-0)^2 + (0.4-1)^2 + (0.3-0)^2 = 1.1$$

Unit 1 wins

Update the weights of the winning unit.

$$\begin{aligned}\text{New Unit 1 weights} &= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + 0.6\,([1\ \ 0\ \ 1\ \ 0] - [0.3\ \ 0.5\ \ 0.7\ \ 0.2])\\ &= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + 0.6\,[0.7\ \ -0.5\ \ 0.3\ \ -0.2]\\ &= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + [0.42\ \ -0.30\ \ 0.18\ \ -0.12]\\ &= [0.72\ \ 0.2\ \ 0.88\ \ 0.08]\end{aligned}$$

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.72 & 0.2 & 0.88 & 0.08\\ 0.6 & 0.7 & 0.4 & 0.3\end{bmatrix}$$

Iteration 2:

Training Sample x_2: (1, 0, 0, 0)

Weight matrix:

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.72 & 0.2 & 0.88 & 0.08\\ 0.6 & 0.7 & 0.4 & 0.3\end{bmatrix}$$

Compute Euclidean distance between x_2: (1, 0, 0, 0) and Unit 1 weights.

$$d^2 = (0.72-1)^2 + (0.2-0)^2 + (0.88-0)^2 + (0.08-0)^2 = 0.74$$

Compute Euclidean distance between x_2: (1, 0, 0, 0) and Unit 2 weights.

$$d^2 = (0.6-1)^2 + (0.7-0)^2 + (0.4-0)^2 + (0.3-0)^2 = 0.9$$

Unit 1 wins

Update the weights of the winning unit:

$$\begin{aligned}\text{New Unit 1 weights} &= [0.72\ \ 0.2\ \ 0.88\ \ 0.08] + 0.6\,([1\ \ 0\ \ 0\ \ 0] - [0.72\ \ 0.2\ \ 0.88\ \ 0.08])\\ &= [0.72\ \ 0.2\ \ 0.88\ \ 0.08] + 0.6\,[0.28\ \ -0.2\ \ -0.88\ \ -0.08]\\ &= [0.72\ \ 0.2\ \ 0.88\ \ 0.08] + [0.17\ \ -0.12\ \ -0.53\ \ -0.05]\\ &= [0.89\ \ 0.08\ \ 0.35\ \ 0.03]\end{aligned}$$

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.6 & 0.7 & 0.4 & 0.3\end{bmatrix}$$

Iteration 3:

Training Sample x_3: (1, 1, 1, 1)

Weight matrix:

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.6 & 0.7 & 0.4 & 0.3\end{bmatrix}$$

Compute Euclidean distance between x_3: (1, 1, 1, 1) and Unit 1 weights.

$d^2 = (0.89 - 1)^2 + (0.08 - 1)^2 + (0.35 - 1)^2 + (0.03 - 1)^2$

$= 2.2$

Compute Euclidean distance between x_3: (1, 1, 1, 1) and Unit 2 weights.

$d^2 = (0.6 - 1)^2 + (0.7 - 1)^2 + (0.4 - 1)^2 + (0.3 - 1)^2$

$= 1.1$

Unit 2 wins

Update the weights of the winning unit:

New Unit 2 weights $= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + 0.6\,([1\ \ 1\ \ 1\ \ 1] - [0.6\ \ 0.7\ \ 0.4\ \ 0.3])$

$= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + 0.6\,[0.4\ \ 0.3\ \ 0.6\ \ 0.7]$

$= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + [0.24\ \ 0.18\ \ 0.36\ \ 0.42] = [0.84\ \ 0.88\ \ 0.76\ \ 0.72]$

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.84 & 0.88 & 0.76 & 0.72\end{bmatrix}$$

Iteration 4:

Training Sample x_4: (0, 1, 1, 0)

Weight matrix:

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.84 & 0.88 & 0.76 & 0.72\end{bmatrix}$$

Compute Euclidean distance between x_4: (0, 1, 1, 0) and Unit 1 weights.

$d^2 = (0.89 - 0)^2 + (0.08 - 1)^2 + (0.35 - 1)^2 + (0.03 - 0)^2$

$= 2.06$

Compute Euclidean distance between x_1: (0, 1, 1, 0) and Unit 2 weights.

$d^2 = (0.84 - 0)^2 + (0.88 - 1)^2 + (0.76 - 1)^2 + (0.72 - 0)^2$

$= 1.3$

Unit 2 wins

Update the weights of the winning unit:

New Unit 2 weights $= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + 0.6\,([0\ \ 1\ \ 1\ \ 0] - [0.84\ \ 0.88\ \ 0.76\ \ 0.72])$

$= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + 0.6\,[-0.84\ \ 0.12\ \ 0.24\ \ -0.72]$

$= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + [-0.5\ \ 0.07\ \ 0.14\ \ -0.43] = [0.34\ \ 0.95\ \ 0.9\ \ 0.29]$

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.34 & 0.95 & 0.9 & 0.29\end{bmatrix}$$

Best mapping units for each of the sample taken are:

x_1: (1, 0, 1, 0) → Unit 1

x_2: (1, 0, 0, 0) → Unit 1

x_3: (1, 1, 1, 1) → Unit 2

x_4: (0, 1, 1, 0) → Unit 2

This process is continued for many epochs until the feature map does not change.

10.9 POPULAR APPLICATIONS OF ARTIFICIAL NEURAL NETWORKS

ANN learning mechanisms are used in many complex applications that involve modelling of non-linear processes. ANN is a useful model that can handle even noisy and incomplete data. They are used to model complex patterns, recognize patterns and solve prediction problems like humans in many areas such as:

1. Real-time applications: Face recognition, emotion detection, self-driving cars, navigation systems, routing systems, target tracking, vehicle scheduling, etc.
2. Business applications: Stock trading, sales forecasting, customer behaviour modelling, Market research and analysis, etc.
3. Banking and Finance: Credit and loan forecasting, fraud and risk evaluation, currency price prediction, real-estate appraisal, etc.
4. Education: Adaptive learning software, student performance modelling, etc.
5. Healthcare: Medical diagnosis or mapping symptoms to a medical case, image interpretation and pattern recognition, drug discovery, etc.
6. Other Engineering Applications: Robotics, aerospace, electronics, manufacturing, communications, chemical analysis, food research, etc.

10.10 ADVANTAGES AND DISADVANTAGES OF ANN

Advantages of ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations of ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.

3. The modelling with ANN is also extremely complicated and the development takes a much longer time.
4. Generally, neural networks require more data than traditional machine learning algorithms, and they do not perform well on small datasets.
5. They are also more computationally expensive than traditional learning techniques.

10.11 CHALLENGES OF ARTIFICIAL NEURAL NETWORKS

The major challenges while modelling a real-time application with ANNs are:

1. Training a neural network is the most challenging part of using this technique. Overfitting or underfitting issues may arise if datasets used for training are not correct. It is also hard to generalize to the real-world data when trained with some simulated data. Moreover, neural network models normally need a lot of training data to be robust and are usable for a real-time application.
2. Finding the weight and bias parameters for neural networks is also hard and it is difficult to calculate an optimal model.

Summary

1. The human nervous system has billions of neurons that are the processing units which make us to perceive things around us, to hear, to see and to smell.
2. A typical biological neuron has four parts called dendrites, soma, axon and synapse. The body of the neuron is called as soma.
3. The neurons form a network structure which processes input information and gives out a response.
4. Artificial neurons are like biological neurons which are called as nodes that can receive one or more input information from other neurons and process the information.
5. The first mathematical model of a biological neuron designed by McCulloch & Pitts receives weighted inputs from other neurons and operates with an activation function.
6. A neuron gets fired when the weighted sum of the received inputs exceeds the threshold of the activation function.
7. An Artificial Neural Network has a network structure represented as a directed graph with a set of neuron nodes and connection links or edges connecting the nodes.
8. The activation function decides whether to fire a neuron or not based on the input signals the neuron receives.
9. Activation functions can be linear or non-linear.
10. Linear functions are generally used in binary perceptrons, where the input values can be classified into any one of the two groups.
11. Non-linear functions are continuous functions that map the input in the range of (0, 1) or (−1, 1), etc.
12. The first neural network model 'Perceptron' is a linear binary classifier used for supervised learning.
13. The Perceptron, designed by Frank Rosenblatt, could actually learn weights and thresholds from training data by itself.

14. A perceptron model can solve any Boolean functions which are linearly separable.
15. XOR is not linearly separable, so a single layer perceptron fails to classify, and hence this problem led to the evolution of a Multi-Layer Perceptron (MLP).
16. Different types of ANNs differ by the network structure, activation function used and the learning rules used. In an ANN, there are three layers called input layer, hidden layer and output layer.
17. An MLP has atleast one hidden layer and as the number of hidden layers is increased, the learning becomes more complex and they form a deep neural network.
18. An MLP uses back propagation for supervised learning of the network.
19. The hidden layer in the RBFNN uses a non-linear radial basis function as the activation function which converts the input parameters into a high dimension space which is then fed into the network to linearly separate the problem.
20. Self-Organizing Feature Map is an unsupervised learning model that clusters data by mapping a high-dimensional data into a two-dimensional map (neurons) or plane.

Key Terms

Scan for *'Additional Key Terms'*



- **Artificial Neural Network (ANN)** – Imitates a human brain which inhibits some intelligence.
- **Neuron** – It is the basic working unit of the brain. These are cells within the nervous system that transmit information to other cells in the body.
- **Soma** – The body of the neuron is called as Soma.
- **Dendrites** – These are the branches of a neuron to accept the input information and process it in the cell body called soma.
- **Axon** –An axon is the outer part of a neuron that passes the processed information from one neuron to another neuron.
- **Synapse** – It is a small junction at the end of a neuron to communicate signals to another neuron.
- **Central Nervous System (CNS)** – It comprises the brain and the spinal cord that controls most of the functions of the body and mind.
- **Peripheral Nervous System (PNS)** – It constitutes of all neurons that lie outside of the central *nervous system* (CNS) and controls the functions of all sensory organs.
- **Artificial Neurons** – These are like biological neurons which are called as nodes that can receive one or more input information and process the information.
- **McCulloch & Pitts Neuron** – It is a simple model of an artificial neuron proposed by Warren McCulloch and Walter Pitts in 1943.
- **Activation Functions** – These are mathematical functions associated with each neuron in the neural network that maps the input signals into output signals.
- **Perceptron** – The first neural network model that is a linear binary classifier used for supervised learning.

- **Multi-Layer Perceptron (MLP)** – It consists of multiple layers with one input layer, one output layer and one or more hidden layers.
- **Feed Forward Neural Network** – It is the simplest neural network that consists of neurons which are arranged in layers and the information is propagated only in the forward direction.
- **Fully Connected Neural Network** – In this network, all the neurons in a layer are connected all other neurons in the next layer.
- **Feedback Neural Network** – It has feedback connections between neurons that allow information flow in both directions in the network.
- **Radial Basis Function Neural Network (RBFNN)** – It is a type of Multi-Layer Perceptron which has one input layer, one output layer and with strictly one hidden layer. The hidden layer uses a non-linear radial basis function as the activation function which converts the input parameters into a high-dimension space, which is then fed into the network to linearly separate the problem.
- **Self-Organizing Feature Map (SOFM)** – It is a special type of Feed Forward Artificial Neural Network used as a competitive learning network for unsupervised learning.

Review Questions

1. Compare biological neuron and artificial neuron.
2. What is the drawback of McCulloch & Pitts mathematical model of an artificial neuron?
3. What are activation functions?
4. List some of the linear and non-linear activation functions.
5. Why XOR problem could not be solved by simple perceptron?
6. Design a two-layer network of perceptron to implement NAND gates. Assume your own weights and biases in the range of [–0.5, 0.5]. Use learning rate $\propto = 0.4$.
7. Discuss about some of the types of artificial neural networks.
8. How a Multi-Layer Perceptron solves the XOR problem. Design an MLP with back propagation to implement the XOR Boolean function.
9. Compare an RBFNN with MLP.
10. Consider the Network architecture with 4 input units and 2 output units. Consider four training samples each vector of length 4.

    Training samples

    $i1$: (1, 1, 1, 0)

    $i2$: (0, 0, 1, 1)

    $i3$: (1, 0, 0, 1)

    $i4$: (0, 0, 1, 0)

    Output Units: Unit 1, Unit 2

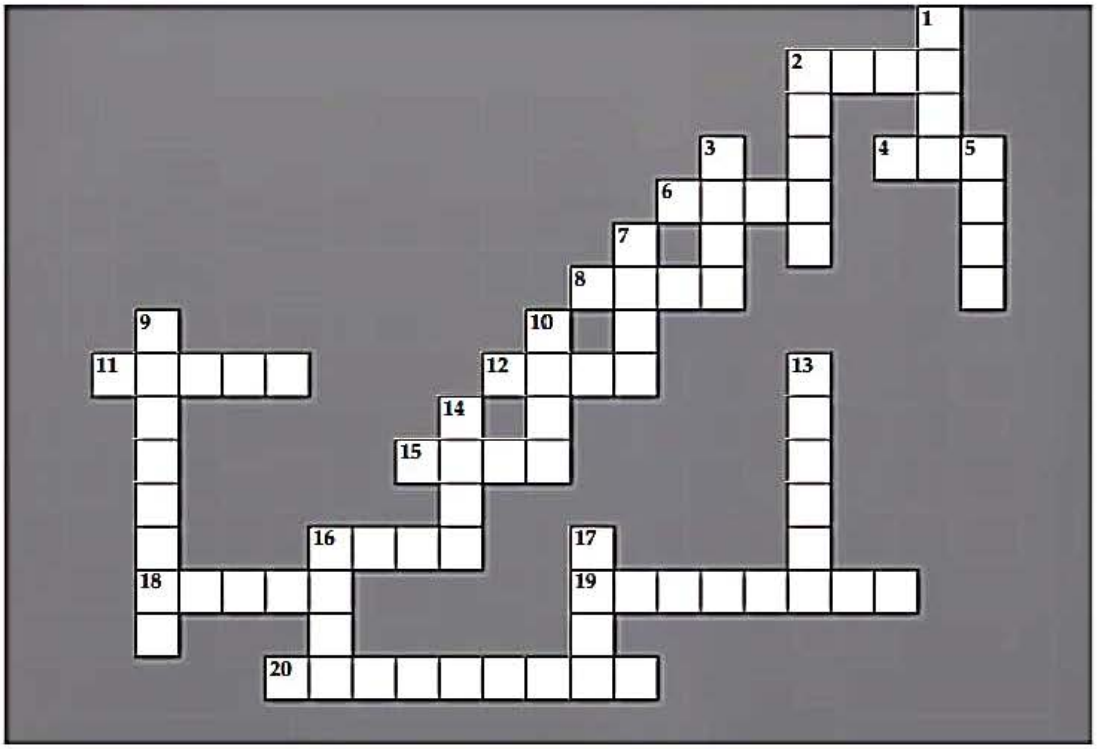
    Learning rate $\eta(t) = 0.6$

    Initial Weight matrix

    $$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix}:\begin{bmatrix}0.2 & 0.8 & 0.5 & 0.1\\ 0.3 & 0.5 & 0.4 & 0.6\end{bmatrix}$$

    Identify an algorithm to learn without supervision? How do you cluster them as expected?

Crossword



Across

2. A typical biological neuron has _____ parts.
4. Perception is made possible by billions of neurons. (Yes/No)
6. Activations decide when to fire a neuron or not. (True/False)
8. Perceptron is a binary linear classifier. (True/False)
11. If all the neurons in a layer are connected, then it is called a _____ connected network.
12. A single layer Perceptron cannot solve the XOR problem. (True/False)
15. Addition of neurons can imporove the complexity. (True/False)
16. RBFFNN uses a kernel that converts input parameters to a higher dimensional space. (True/False)
18. Axon is _____ (inner/outer) part of the neuron that passes information.
19. Neurons form a neural architecture and process information to give a _____.
20. The branches of the neuron are called _____.

Down

1. A graph can be used to structure a neural network. (True/False)
2. When the weighted sum of the inputs exceeds a threshold value, the neuron gets _____.
3. A binary perceptron can be a linear function. (True/False)
5. The body of the neuron is called _____.
7. Frank Rosenblatt designed a perceptron. (True/False)
9. Synapse is a smaller _____ at the end of the neuron.
10. Perceptron can solve all the problems that are linearly separable. (True/False)
13. McCulloch and Pitts designed a first mathematical model of a _____.
14. MLP uses supervised learning. (True/False)
16. SOM is used to cluster data. (True/False)
17. The central nervous system comprises brain and spinal cord. (True/False)

Word Search

Find and mark the words listed below.

A	C	Q	I	I	P	B	V	S	K	Q	G	H	T	H	R	F	M	Q	Q	L	E	W	U	N
O	U	T	E	R	H	E	C	L	Z	E	I	J	S	R	X	Z	Q	N	X	D	Y	Y	T	N
H	R	W	V	M	F	N	T	R	U	E	Y	U	H	K	U	B	Q	O	R	H	O	L	K	O
T	A	S	B	E	N	O	V	F	V	D	U	N	G	U	X	E	H	C	E	T	Y	F	M	R
R	M	I	R	Y	G	S	U	J	M	Z	W	C	X	W	F	Q	J	B	S	P	H	E	X	Q
U	M	G	S	K	K	R	V	R	M	D	L	T	Z	I	T	R	U	E	P	N	K	O	S	F
E	H	Z	P	O	O	Z	O	J	Q	C	W	I	R	B	V	S	Z	S	O	E	M	I	F	K
V	E	X	U	O	M	A	J	H	R	D	Z	O	E	L	E	E	Z	A	N	B	V	T	U	L
Z	O	C	I	R	G	A	J	R	F	T	L	N	S	T	Q	R	W	H	S	B	W	H	L	K
U	Q	G	T	N	L	U	F	Y	U	R	U	U	I	D	Z	Y	G	K	E	P	O	F	L	L
F	V	V	B	T	T	V	Y	N	B	U	F	R	N	E	U	R	O	N	L	X	T	O	Y	I
P	K	R	B	A	R	C	C	K	C	E	D	I	H	U	R	P	P	G	Q	T	R	Y	T	I
J	Q	A	R	E	U	U	W	Q	F	N	E	T	R	A	S	T	O	V	V	D	U	Q	R	R
I	T	Y	U	U	E	K	E	X	E	U	K	J	J	E	E	N	Z	U	A	K	E	W	U	B
B	X	R	U	P	T	H	H	D	R	H	K	T	I	S	D	W	V	J	I	L	F	K	E	R
Z	T	L	U	T	R	T	O	T	A	O	Q	B	I	X	A	W	C	G	I	A	Z	U	V	T
E	E	O	V	E	F	N	D	O	Z	P	J	M	B	C	U	J	R	F	E	S	J	X	K	X

Yes	Four	Response	Neuron	Fired	True	True	True
True	True	True	True	True	True	True	True
Soma	Dendrites	Outer	Junction	True	Fully		